Human Pose Estimation

1. Experiment Objective

Use an OpenCV deep learning model to detect human pose keypoints in the camera image and visualize them in real time by connecting them into a skeleton.

2. Experiment Principle

1. Key Terms

Term	Description
Human Pose Estimation	Detects human keypoints (joints, facial features) in an image using a deep learning model, and visualizes the pose by connecting them into a skeleton
OpenPose	A classic bottom-up human pose estimation algorithm that first locates all keypoints and then assembles them into a skeleton
DNN	A pre-trained TensorFlow-based model that extracts features from images and infers keypoint locations
Keypoints	The 18 core body joints (nose, neck, shoulder, elbow, wrist, hip, knee, ankle, eye, ear), plus a background channel for a total of 19
Confidence	The model's confidence score for each keypoint location; keypoints below the threshold of 0.2 are discarded
Skeleton Lines	17 keypoint connection relationships predefined from human anatomy, used to visualize the skeleton

2. Technical Architecture

```
Terminal 1:  start agent + camera + joint models
Terminal 2:  start human pose estimation
```

Data Flow: ESP32 camera → `/camera/image_raw` → `imgmsg_to_cv2` image conversion → `blobFromImage` converts to a 368×368 blob → OpenPose MobileNet forward inference → 19-channel heatmaps → confidence threshold filtering → draw keypoints + skeleton lines → `cv2.imshow` real-time display.

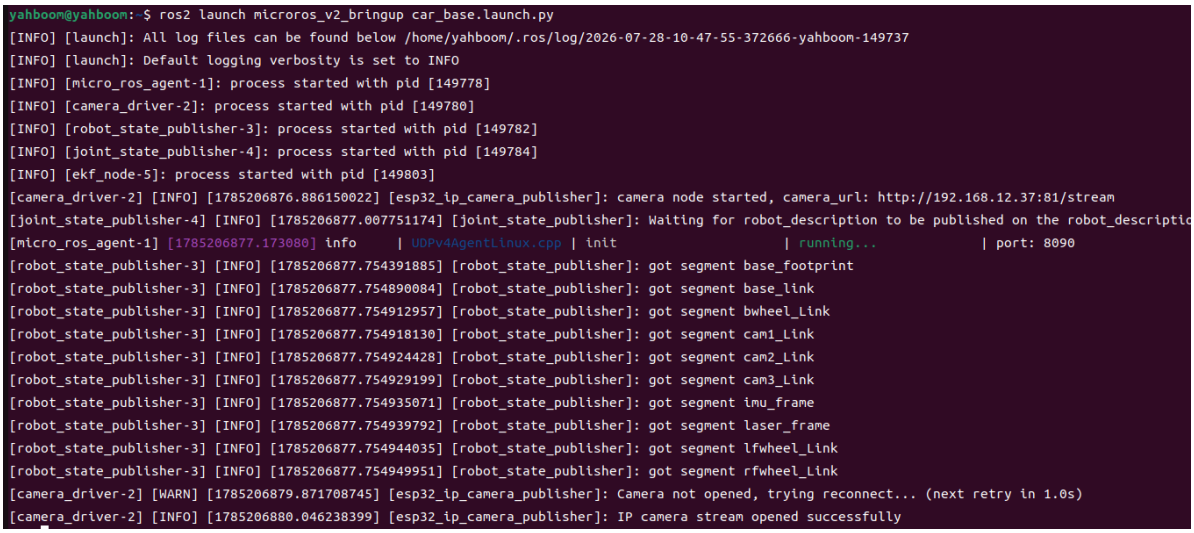
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☑ Supported
No-PTZ Version	☑ Supported
Required Peripherals	ESP32 camera

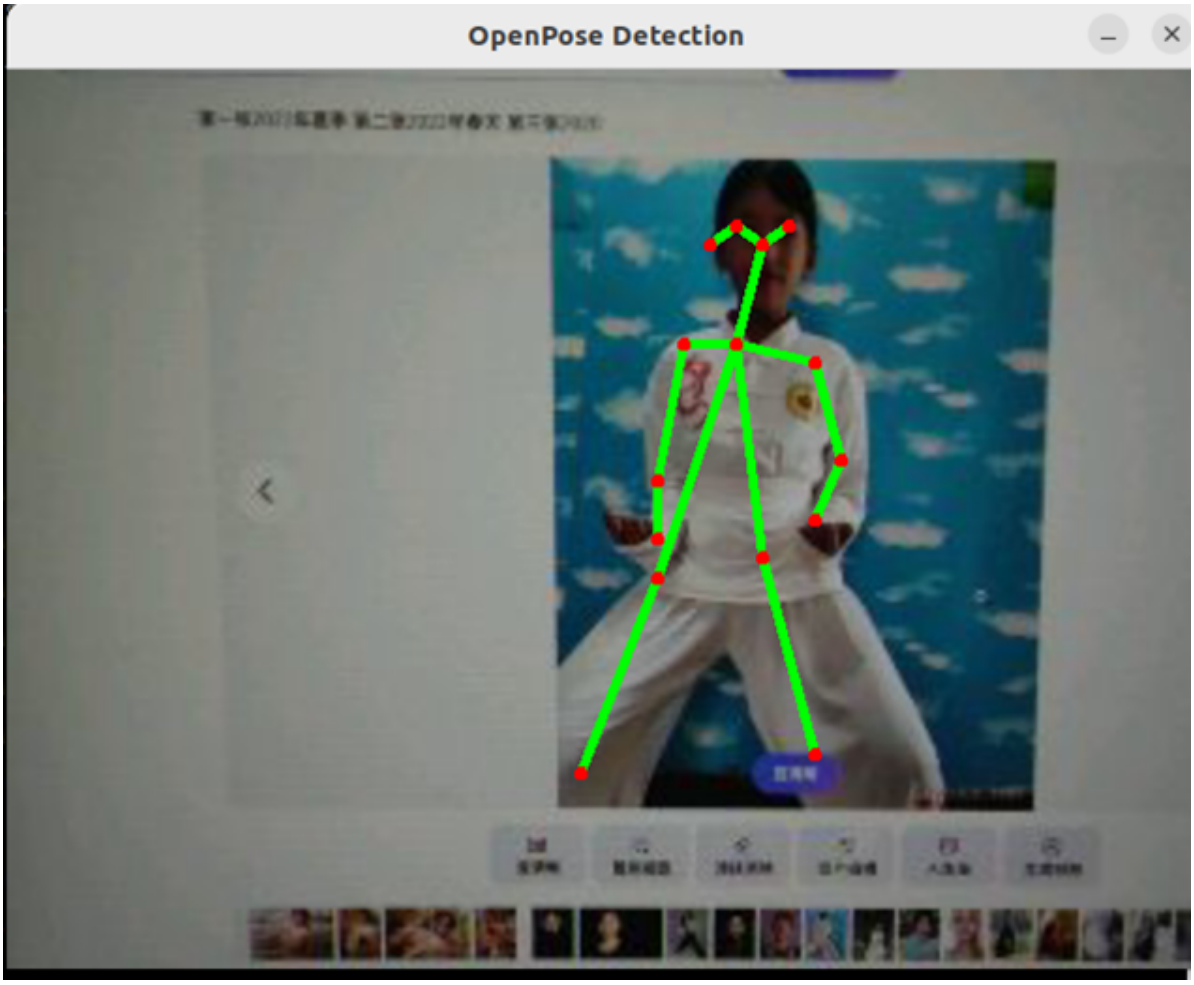
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start Human Pose Estimation (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_pose.launch.py
```



4. Operation Instructions

Workflow: The node subscribes to `/camera/image_raw` → image conversion → OpenPose inference (368×368 blob) → heatmaps → keypoints with confidence > 0.2 are kept → red dots mark keypoints and green lines draw the skeleton connections → `cv2.imshow` real-time display.

Keypoint Mapping (19 body parts):

Index	Part	Index	Part	Index	Part
0	Nose	7	LWrist	14	REye
1	Neck	8	RHip	15	LEye
2	RShoulder	9	RKnee	16	REar
3	RElbow	10	RAnkle	17	LEar
4	RWrist	11	LHip	18	Background (not displayed)
5	LShoulder	12	LKnee		
6	LElbow	13	LAnkle		

The 17 skeleton connections cover the whole body: torso (neck → shoulder/hip), limbs (shoulder → elbow → wrist, hip → knee → ankle), and head (nose → eye → ear, neck → nose).

Interaction:

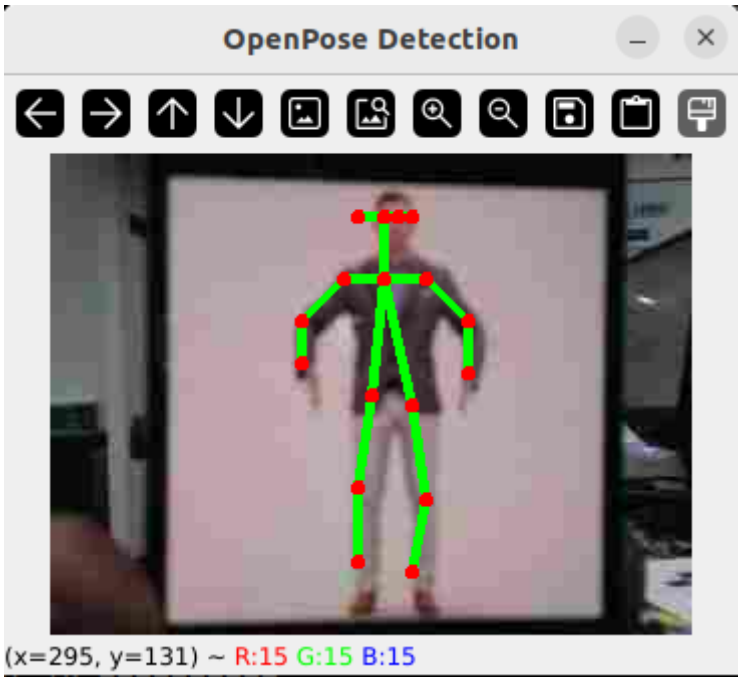
Key	Function
`q` / `ESC`	Exit the program and close the OpenCV window

6. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **Normal detection**: When a person is in the frame, the 19 keypoints are marked with red filled circles and the 17 skeleton connections are overlaid in real time with green lines
- **No-person scene**: All keypoint confidences are below 0.2, so the raw image is shown without annotation
- **Partial occlusion**: Only keypoints with confidence > 0.2 are displayed; occluded parts are not drawn
- **Multiple people**: This is single-person pose estimation; with multiple people, only the person with the highest confidence may be detected



5. Program Analysis

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_pose.launch.py`
> - Node: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_pose.py`

1. Core Node Analysis

`OpenPoseDetectionNode` subscribes to the raw image topic, decodes the frames, feeds them into the OpenPose MobileNet model for inference, and overlays skeleton annotations on the image.

Constructor (`__init__`)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/de
tect_pose.py
class OpenPoseDetectionNode(Node):
    def __init__(self):
        super().__init__('openpose_detection_node')
        self.bridge = CvBridge()
        topic_name = '/camera/image_raw'

        self.subscription = self.create_subscription(
            Image, topic_name, self.image_callback,
            QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                       reliability=QoSReliabilityPolicy.BEST_EFFORT,
                       durability=QoSDurabilityPolicy.VOLATILE))

        # Load the OpenPose model
        self.BODY_PARTS = {
            "Nose": 0, "Neck": 1, "RShoulder": 2, "RElbow": 3, "RWrist": 4,
            "LShoulder": 5, "LElbow": 6, "LWrist": 7, "RHip": 8, "RKnee": 9,
            "RAnkle": 10, "LHip": 11, "LKnee": 12, "LAnkle": 13, "REye": 14,
            "LEye": 15, "REar": 16, "LEar": 17, "Background": 18}
        self.POSE_PAIRS = [
            ["Neck", "RShoulder"], ["Neck", "LShoulder"], ...]

        self.net = cv.dnn.readNetFromTensorflow(
            ".../config/graph_opt.pb")
```

Key logic notes:

- The image subscription uses `BEST_EFFORT` QoS: video streaming allows frame drops, and `depth=1` ensures only the latest frame is kept
- The OpenPose model file `graph_opt.pb` is a pre-trained TensorFlow MobileNet version; its small size suits embedded scenarios
- The `BODY_PARTS` dictionary defines the indices of the 19 body parts, and the `POSE_PAIRS` list defines the 17 skeleton connection relationships

Image Callback (`image_callback`)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/de
tect_pose.py
def image_callback(self, msg):
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    except Exception as e:
        self.get_logger().error(f'Image decode failed: {e}')
        return

    frame = self.openpose(frame)
```

```
        cv.imshow('OpenPose Detection', frame)

        if cv.waitKey(1) & 0xFF == ord('q'):
            self.get_logger().info("Exiting the application...")
            rclpy.shutdown()
```

Key logic notes:

- `imgmsg_to_cv2` converts the `sensor_msgs/Image` raw image frame into a BGR ndarray
- Decoding failures do not crash the node: after logging an error, it `return`s to skip the current frame and wait for the next one
- OpenPose inference runs in `self.openpose(frame)`, which returns the image with skeleton annotations overlaid
- `cv2.waitKey(1)` both drives the OpenCV GUI event loop and detects the exit key

2. Key Parameters

> Parameter definition file: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_pose.py`

Parameter	Type	Default Value	Description
`topic_name`	string	`/camera/image_raw`	Input image topic
`confidence_threshold`	float	`0.2`	Keypoint confidence threshold; keypoints below this value are not displayed
`blob_size`	tuple	`(368, 368)`	Model input blob size (width × height)
`input_model`	string	`graph_opt.pb`	OpenPose MobileNet TensorFlow model file

3. Node Description

Node	Function
`openpose_detection_node`	Subscribes to raw images, runs OpenPose DNN inference for human keypoints, and visualizes with OpenCV

6. Frequently Asked Questions

Problem	Cause	Solution
Black window	No data on the camera topic	Check with `ros2 topic echo /camera/image_raw`
Cannot detect a person	Insufficient light or distance too far	Adjust the lighting and move the person closer to the camera